# X-Wing Hybrid Key Encapsulation and Parallel Performance Study

**Dhananjay Bhattacharya**

Birla Institute of Technology and Science, Pilani, K. K. Birla Goa Campus, India

***Abstract*—**This report documents an educational implementation and parallel-computing study of the X-Wing hybrid key encapsulation mechanism (KEM). The project preserves the original paper construction as a Python/OpenSSL command-line baseline and separately implements the September 2026 X-Wing Internet-Draft revision 11 in native C through OpenSSL EVP. The repository also includes a local browser dashboard, a KEM-to-HKDF-to-AES-GCM demonstration, an OpenMP batch-throughput harness, a research-only ciphertext-hashing comparator, and an isolated number-theoretic transform (NTT) laboratory. This report explains the version boundaries, data flow, software architecture, and evaluation methodology. It distinguishes completed source-code work from future recommendations. No build, test, demo, or benchmark was run while preparing this report; therefore, it presents no new performance or correctness measurements. The NTT experiment is a teaching workload and does not implement ML-KEM polynomial arithmetic. The code is a research prototype and has not received an independent security audit.

***Index Terms*—**Hybrid KEM, X-Wing, ML-KEM-768, X25519, OpenMP, NTT, parallel computing, cryptographic engineering.

## I. INTRODUCTION

X-Wing combines ML-KEM-768, a lattice-based post-quantum key-encapsulation mechanism, with X25519, a classical elliptic-curve Diffie–Hellman primitive. The hybrid secret is intended to remain useful if either component later proves weaker than expected, subject to the construction's assumptions and correct implementation [1], [2]. This project turns that design into a readable baseline and a separate set of systems experiments.

The repository has two cryptographic targets. The Python implementation follows Figure 12 of the original X-Wing paper. The native C library follows X-Wing Internet-Draft revision 11. These targets use related primitives and have similar sizes, but their key derivation, private-key representation, and combiner byte order differ. They are not presented as interoperable.

The parallel-computing work measures independent KEM encapsulations across OpenMP worker counts and explores transform optimizations in a separate NTT lab. The source includes instrumentation, but no measurements have been collected for this report. The contribution reported here is the implementation and evaluation framework.

## II. BACKGROUND AND RELATED WORK

### *A. Hybrid Key Encapsulation*

A KEM has key generation, encapsulation, and decapsulation operations. Encapsulation uses a recipient public key to produce a ciphertext and shared secret; decapsulation uses the corresponding private key and ciphertext to recover that secret. X-Wing combines an ML-KEM post-quantum component and an X25519 classical component using a hash-based combiner [1], [2].

### *B. Specification Versions*

The original paper's Figure 12 uses separately generated ML-KEM-768 and X25519 keys. Its six-byte label is 5c2e2f2f5e5c. Its combiner hashes label || ss_M || ss_X || ct_X || pk_X. The ML-KEM ciphertext is omitted from this construction's combiner input. The Python path preserves this ordering.

Native C targets draft-connolly-cfrg-xwing-kem-11, dated 23 September 2026. Revision 11 derives key material from a 32-byte seed using SHAKE256 and defines the combiner input as ss_M || ss_X || ct_X || pk_X || label [2]. Moving the label changes the byte sequence. Matching lengths do not make the formats interoperable.

### *C. Standards and Deployment Context*

ML-KEM-768 is specified in NIST FIPS 203 [3]. OpenSSL added ML-KEM support to its EVP interfaces in version 3.5, allowing the native implementation to call a maintained primitive implementation [4]. For TLS, RFC 10024 defines hybrid ECDHE-MLKEM groups, including X25519MLKEM768 [5]. This protocol-specific TLS key agreement is distinct from standalone X-Wing.

Deployment also involves message size and network compatibility. Chrome's hybrid Kyber deployment reported that some middleboxes failed when larger ClientHello messages met fixed-size assumptions [6]. This motivates measuring compatibility and message sizes alongside local primitive latency.

## III. PROJECT SCOPE AND VERSION BOUNDARIES

This is a teaching and performance-research project. It includes the paper baseline, a draft-11 native implementation, a local key-use demonstration, parallel experiments, and an explanatory frontend. It does not implement TLS, production key management, authenticated peer identity, a production network protocol, a certification workflow, or a constant-time audit. Operating-system scheduler and performance-counter work remains out of scope.

| Path | Target and purpose | Boundary |
|---|---|---|
| Python baseline | Original paper composition via OpenSSL CLI; dashboard and round-trip. | Paper ordering; process timing includes startup and file I/O. |
| Native C library | X-Wing draft-11 via OpenSSL EVP. | Separate target; requires OpenSSL 3.5 or newer. |
| Hash-CT mode | Research comparator that appends ML-KEM ciphertext to hash input. | Not standard X-Wing and not interoperable. |
| NTT laboratory | Cyclic convolution and transform-optimization study. | Teaching ring; not FIPS 203 arithmetic; not linked to KEM. |

Every test result and chart should name the paper, draft-11, or hash-CT mode. An output from one mode must not be treated as a known-answer vector for another.

## IV. SYSTEM ARCHITECTURE

### *A. Python Paper Baseline*

The Python path separates primitive operations from composition. xwing/openssl_backend.py invokes OpenSSL CLI operations and checks output sizes. xwing/paper_baseline.py owns the paper's operation order and combiner. The CLI emits JSON, while the dashboard server calls the same core rather than simulating cryptography in JavaScript.

The dashboard binds to 127.0.0.1 and exposes health and run endpoints. It walks through recipient key generation, ML-KEM encapsulation, ephemeral X25519 derivation, combination, and receiver verification. It shows sizes, timings, public-data previews, and short fingerprints without exposing private keys or raw secrets. The browser is a view layer.

### *B. Native Draft-11 Path*

native/src/xwing.c uses OpenSSL EVP for ML-KEM-768, X25519, SHAKE256 seed expansion, and SHA3-256. Its fixed-size interface uses a 32-byte seed, 1,216-byte public key, 1,120-byte ciphertext, and 32-byte shared secret. The API is demonstration-oriented and is not a production length-aware interface.

native/src/demo.c exercises draft-11 and a separately labeled hash-CT research mode. native/src/secure_demo.c derives a key from the KEM result with HKDF-SHA256, then encrypts and decrypts a fixed local message with AES-256-GCM. This shows key use and symmetric ciphertext authentication; it does not authenticate the recipient public key or peer identity.

### *C. Parallel and Reporting Utilities*

native/src/benchmark.c measures independent encapsulations against one recipient public key. It emits CSV with elapsed time, throughput, speedup, and parallel efficiency, and writes compiler, OpenSSL, and processor-count metadata to standard error. scripts/plot_results.py is intended to turn saved KEM or NTT CSV into figures. Reproducibility records should also include compiler flags, hardware, provider configuration, and run parameters.

## V. ALGORITHM AND IMPLEMENTATION METHODOLOGY

### *A. Paper Construction*

Key generation creates an ML-KEM-768 keypair and a static X25519 keypair. Encapsulation obtains (ct_M, ss_M) from ML-KEM and generates an ephemeral X25519 scalar. The sender publishes ct_X, the ephemeral public key, and derives ss_X using the recipient public key. Both endpoints hash the same 134 bytes: a six-byte label, 32-byte ML-KEM secret, 32-byte X25519 secret, 32-byte X25519 ciphertext, and 32-byte recipient X25519 public key. Decapsulation reconstructs the component secrets and digest [1].

The paper path's logical public key is 1,216 bytes (1,184 + 32); its ciphertext is 1,120 bytes (1,088 + 32). The final shared secret is 32 bytes. Comparing sender and receiver outputs for a generated valid input is functional evidence, not an independent conformance test or a proof of security.

### *B. Draft-11 Construction*

The native path expands a 32-byte X-Wing seed with SHAKE256 into ML-KEM and X25519 material. It serializes the ML-KEM public key before the 32-byte X25519 public key. The ciphertext contains the 1,088-byte ML-KEM ciphertext and the 32-byte X25519 ephemeral public value. Both endpoints combine ss_M, ss_X, ct_X, pk_X, and the label in draft-11 order to produce a 32-byte SHA3-256 result [2].

The paper and draft-11 targets have equal public-key, ciphertext, standard combiner-input, and output lengths: 1,216, 1,120, 134, and 32 bytes. Equal lengths do not establish byte compatibility. The research hash-CT mode adds the 1,088-byte ML-KEM ciphertext to the 134-byte input, for 1,222 bytes, only as a hashing-cost comparator.

### *C. Application Demonstration*

The native secure demo feeds the draft-11 result to HKDF-SHA256 with a fixed context string and derives a 32-byte AES key. AES-256-GCM uses a fresh 12-byte nonce and a 16-byte tag. The demo checks that decryption recovers the local message. Since key generation, encapsulation, and decryption run in one process, it is an API demonstration rather than a secure communications protocol.

## VI. PARALLEL COMPUTING AND NTT OPTIMIZATIONS

### *A. OpenMP KEM Batch Scaling*

Encapsulations for one recipient are independent. The benchmark places an OpenMP parallel-for across a fixed operation count with schedule(static). Each iteration owns its ciphertext and secret buffers. This coarse-grained strategy avoids shared per-operation output memory and amortizes worker overhead over many operations. The harness sweeps powers-of-two worker counts up to the available OpenMP processor count and compares draft-11 with hash-CT.

For p workers and elapsed time T_p, throughput is N/T_p, speedup is S_p = T_1/T_p, and efficiency is E_p = S_p/p. Future runs should repeat configurations, separate warm-up from measured work, report median and spread, and record OpenSSL/provider and machine details. This defines the measurement protocol; no values were collected here.

### *B. NTT Laboratory*

The NTT lab compares quadratic naive cyclic convolution with a radix-2 length-256 transform over the finite field q = 12,289. It precomputes forward and inverse twiddles, uses iterative butterfly stages after bit reversal, and applies Barrett modular reduction. Source includes a pre-timing check comparing 16 deterministic input pairs against the naive implementation. That check was not run for this report.

The lab implements batch-level OpenMP across independent polynomial multiplications and stage-level OpenMP across butterfly blocks. Stage mode may open parallel regions repeatedly. Batch-level parallelism is the natural first candidate because each task contains a full multiplication. Synchronization and thread-launch overhead may outweigh the work in a 256-point transform. This remains a hypothesis pending measurement.

The lab uses cyclic convolution modulo X^256−1 over F_12289. FIPS 203 ML-KEM arithmetic instead uses a degree-256 ring modulo 3,329 with a negacyclic polynomial relation [3]. The experiment is isolated from both KEM paths. An exact FIPS 203 arithmetic study belongs in a separately validated module.

### *C. Suggested OpenMP Refinements*

- Keep operation-level parallelism as the baseline. Increase batch size, reuse initialized key material, and keep per-operation state thread-local. Compare scheduling modes only if workload variation or scheduling cost is measured.
- Run serial and parallel configurations with the same build and operation count. Include repeated trials, warm-up, median, variation, throughput, speedup, and efficiency. Stop increasing worker count when throughput flattens.
- Separate primitive work from process startup, allocation, file operations, serialization, and frontend rendering. Python CLI timings include process launch and file I/O; use the native harness for the OpenMP study.
- Measure OpenSSL/provider threading behavior before nesting OpenMP around a library that may use internal workers. Record OMP_NUM_THREADS, binding, and oversubscription conditions.
- For NTT, compare serial NTT, batch OpenMP, and stage OpenMP on identical inputs. Retain the precomputed plan; only remove per-stage parallel-region overhead or add SIMD after measurement and reference comparison.
- Do not move experimental arithmetic into the cryptographic path until version-specific known-answer vectors and cross-library conformance checks pass.

| Metric | Definition | Meaning |
|---|---|---|
| Throughput | N / T_p | Operations completed per second. |
| Speedup | T_1 / T_p | Parallel runtime relative to one worker. |
| Efficiency | (T_1 / T_p) / p | Speedup normalized by worker count. |
| Variation | Repeated-trial spread | Stability and measurement noise. |

A useful benchmark record includes CPU model and core counts, memory, OS, compiler and flags, OpenSSL/provider, OpenMP runtime, thread affinity, operation count, repetitions, and raw CSV. The current fixed-recipient encapsulation benchmark is a throughput experiment, not a complete deployment study.

## VII. FRONTEND AND LEARNING EXPERIENCE

The browser frontend explains the real Python backend as a sequence: recipient key publication, two encapsulation branches, hash combination, and receiver reconstruction. It shows component sizes and a secret fingerprint without disclosing the secret. It explains why the paper construction omits the ML-KEM ciphertext from its combiner and visibly separates the paper path from draft-11.

The interface links the KEM flow to the key-use demo and future research such as cross-library interoperability and a true ML-KEM NTT. It is local-only and should not be exposed as a public service.

## VIII. IMPLEMENTATION STATUS AND RESULTS

The repository contains source for the paper baseline, native draft-11 KEM, research hash-CT comparison, local AEAD demonstration, OpenMP benchmarks, NTT lab, plotting utility, dashboard, running instructions, threat model, test plan, benchmark methodology, and versioning notes. Tests and demos exist as source artifacts; their presence is not a passing execution.

No build, test, dashboard run, demo, correctness check, or benchmark was performed while preparing this report, at the user's request. There are no project-generated latency, speedup, efficiency, correctness, or portability results to claim. The NTT consistency check and KEM round-trip comparisons remain unexecuted in this reporting pass. The X-Wing paper reports 70,942/139,374/116,930 cycles for keypair/encapsulation/decapsulation; X-Wing-Hash-CT reports 70,976/150,708/128,336. These AVX2 paper results are not project measurements [1].

The 134-byte and 1,222-byte combiner-input lengths are derived from the defined field sizes. They describe input encoding, not performance. Future plots should be generated from preserved raw CSV and state the machine and library versions.

## IX. SECURITY AND LIMITATIONS

This project uses OpenSSL components but is not a production cryptographic library. The native fixed-size buffer API, demonstration-oriented error handling, and CLI subprocess layer require further engineering before integration. The code has not received a cryptographic audit, constant-time assessment, fuzzing, formal verification, or cross-implementation known-answer testing.

A KEM establishes shared key material; it does not authenticate peer identity. AES-GCM authenticates a ciphertext under the derived key, but an attacker able to substitute the recipient public key may still cause a key to be established with the wrong peer. Use an authenticated protocol such as TLS or HPKE for real communication. The local dashboard must remain bound to loopback.

Draft-11 is an Internet-Draft and may change [2]. Hash-CT is a nonstandard research mode. Native EVP use requires OpenSSL 3.5 or newer. Python subprocess timing includes substantial non-primitive overhead. The teaching NTT is neither ML-KEM arithmetic nor a replacement for vetted ML-KEM.

## X. RECOMMENDED ADD-ONS

### *A. Cross-Library Interoperability and Known-Answer Tests*

The highest-value next milestone is deterministic vectors for each version and comparison with independent X-Wing implementations, including Cloudflare CIRCL and Google's BoringSSL API [2]. Keep paper and draft-11 vectors separate. Add malformed encodings and error cases. A same-implementation round trip cannot establish byte-for-byte interoperability.

### *B. TLS Compatibility Study*

Build a separate experiment using RFC 10024's X25519MLKEM768 group rather than inserting standalone X-Wing into TLS [5]. Measure negotiation success, key-share and ClientHello sizes, latency, fallback behavior, and failures through representative middleboxes. Chrome's deployment shows that packet-size assumptions can affect rollout [6].

### *C. Exact ML-KEM Arithmetic and Deployment Measurements*

Implement FIPS 203's ring and NTT representation in an isolated module. Compare it with a naive reference and specification test vectors before studying Barrett or Montgomery reduction, SIMD, or batch OpenMP [3]. A deployment study should measure end-to-end handshake and packet effects rather than infer service performance from isolated KEM calls.

### *D. Authentication and Key Management*

Extend the AEAD demo as a separate protocol study: authenticate or pin recipient keys, define message framing and nonce rules, bind context into the key schedule, and demonstrate the effect of key substitution. Compare the result with a standard protocol; do not describe it as a TLS replacement.

## XI. CONCLUSION

The project organizes an original-paper baseline, a separate native draft-11 implementation, a KEM-to-AEAD demonstration, OpenMP throughput work, a standalone NTT lab, and an explanatory frontend in one documented codebase. The methodology preserves version boundaries and delegates primitives to OpenSSL. This is an implementation and evaluation framework, not a measured performance report. The next step should establish version-specific known-answer vectors and cross-library agreement before broader optimization or deployment experiments.

## REFERENCES

[1] M. Barbosa, D. Connolly, J. D. Duarte, A. Kaiser, P. Schwabe, K. Varner, and B. Westerbaan, “X-Wing: The Hybrid KEM You've Been Looking For,” IACR Communications in Cryptology, vol. 1, no. 1, 2024, doi: 10.62056/a3qj89n4e. [Online]. Available: https://eprint.iacr.org/2024/039

[2] D. Connolly, P. Schwabe, and B. Westerbaan, “X-Wing: General-Purpose Hybrid Post-Quantum KEM,” Internet-Draft draft-connolly-cfrg-xwing-kem-11, Sep. 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-connolly-cfrg-xwing-kem-11

[3] National Institute of Standards and Technology, Module-Lattice-Based Key-Encapsulation Mechanism Standard, FIPS PUB 203, Aug. 2024, doi: 10.6028/NIST.FIPS.203. [Online]. Available: https://csrc.nist.gov/pubs/fips/203/final

[4] OpenSSL Project, “EVP_PKEY-ML-KEM” and “EVP_KEM-ML-KEM,” OpenSSL 3.5 Documentation. [Online]. Available: https://docs.openssl.org/3.5/man7/EVP_PKEY-ML-KEM/ and https://docs.openssl.org/3.5/man7/EVP_KEM-ML-KEM/

[5] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, and D. Stebila, “Post-Quantum Traditional (PQ/T) Hybrid Key Agreement Mechanisms for TLS 1.3,” RFC 10024, Aug. 2026. [Online]. Available: https://datatracker.ietf.org/doc/rfc10024/

[6] Chromium Security Team, “Protecting Chrome Traffic with Hybrid Kyber KEM,” Chromium Blog, Aug. 2023. [Online]. Available: https://blog.chromium.org/2023/08/protecting-chrome-traffic-with-hybrid.html